

Software Architecture Document (SAD)

RetailSync: Centralized Super Shop Warehouse Management System

Subtitle: Technical Architecture, Component Decomposition, Concurrency Isolation, and Data Flow Design

Course: SE-231 (Software System Analysis & Design / Capstone Project 2)

Academic Institution: Daffodil International University (DIU), Department of Software Engineering

Author: Raisul Islam Likhon (Section: SWE-44D)

Date: September 2026 | Version: 1.0.0-RELEASE

1. Architectural Principles & System Goals

RetailSync is engineered as a **decoupled, multi-tier cyber-physical software platform** optimized for transactional reliability, sub-second latency, and rigorous data consistency:

- ACID-First Relational Ledger:** Financial transactions and stock balance state transitions are non-negotiable. Every stock movement is logged in an append-only double-entry ledger within strict database transaction boundaries.
- Sub-2.0s Concurrency Guarantee:** Retail POS checkout counters must never wait on warehouse database contention. High-throughput row-level locking (**SELECT ... FOR UPDATE**) guarantees zero deadlocks and zero phantom over-selling.
- Decoupled Stateless Services:** Backend business domain services remain stateless, containerized via Docker, allowing horizontal scaling behind reverse proxies and connection poolers.
- Offline-Tolerant Client Edges:** Network outages in steel-reinforced warehouse racking or retail branches are mitigated via client-side Service Workers and encrypted IndexedDB queues with idempotent sync replay.
- Defense-in-Depth Security:** End-to-end TLS 1.3 encryption, Argon2id password hashing, JWT stateless access tokens with rotated refresh tokens, and immutable audit logs.
- Hybrid Enterprise Edge Extensibility:** Native architectural adapters allow plugging in automated stationary dock gate scanners (ESP32 MCU + RFID / fixed industrial scanners) for bulk pallet intake.

2. High-Level 4-Tier System Architecture

```
graph TD
  subgraph Tier1["1. PHYSICAL EDGE & DATA CAPTURE LAYER"]
    A1["Handheld Barcode Scanners<br/>(Bluetooth HID 1D/2D)"]
    A2["Mobile Smartphone Cameras<br/>(PWA ZXing / Html5-QRCode)"]
    A3["Retail POS Cash Registers<br/>(PrismPOS / Web POS Client)"]
    A4["Enterprise Edge Gate (Optional)<br/>(ESP32 + RFID over MQTT/TLS)"]
  end

  subgraph Tier2["2. API GATEWAY & INGESTION LAYER"]
    B1["Nginx Reverse Proxy & Load Balancer<br/>(SSL/TLS 1.3 Termination, Rate Limiting)"]
    B2["Stateless API Gateway<br/>(JWT Auth, Request Validation, CORS)"]
    B3["MQTT Ingestion Broker<br/>(Mosquitto/EMQX with TLS & ACLs)"]
  end
```

end

subgraph Tier3["3. BUSINESS APPLICATION & ENGINE LAYER"]

C1["Inbound Receiving & GRN Service"]
 C2["Directed Spatial Putaway Engine"]
 C3["Real-Time FEFO/FIFO Ledger Service"]
 C4["Sub-2.0s POS Concurrency Controller"]
 C5["Replenishment DSS (EOQ / Greasley)"]
 C6["Outbound Wave & Pickpath Dispatch"]
 C7["Cycle Counting & Anomaly Detection"]

end

subgraph Tier4["4. PERSISTENCE & DATA STORAGE LAYER"]

D1["PostgreSQL 16 Core Relational DB
3NF Schema, ACID Ledger, B-Tree Indexes"]
 D2["Redis 7 In-Memory Cache
Session Store, Distributed Locks, Rate Limits"]
 D3["Write-Ahead Logs (WAL) & PITR Storage
Encrypted Cloud Object Storage / S3"]

end

A1 --> |Bluetooth / HTTPS| B1
 A2 --> |HTTPS REST| B1
 A3 --> |HTTPS POS Sync API| B1
 A4 --> |MQTT Port 8883| B3

B1 --> B2
 B3 --> C1
 B2 --> C1
 B2 --> C2
 B2 --> C3
 B2 --> C4
 B2 --> C5
 B2 --> C6
 B2 --> C7

C1 & C2 & C3 & C4 & C5 & C6 & C7 --> D1
 C4 & C2 --> D2
 D1 -.-> |Continuous Archival| D3

3. Component & Service Decomposition

3.1 Service Catalog & Responsibilities

Service Identifier	Technology & Framework	Primary Architectural Responsibilities	Inter-Service Communication
Auth & Security Service	Node.js (TypeScript) / Python FastAPI	JWT token generation, refresh token rotation, Argon2id password hashing, RBAC permission evaluation.	Synchronous REST / Internal Middleware

Service Identifier	Technology & Framework	Primary Architectural Responsibilities	Inter-Service Communication
Product Master Service	Python FastAPI / Pydantic v2	Master SKU catalog, EAN-13 barcode validation, ABC turnover classification, temperature requirements.	Synchronous REST, Redis Cache
Inbound & GRN Service	Python FastAPI / SQLAlchemy 2.0	Purchase Order matching, carton scan verification, damage quarantine segregation, digital GRN generation.	REST / PostgreSQL Transactions
Spatial Putaway Engine	Python FastAPI / NetworkX	Dynamic Zone-Aisle-Rack-Shelf-Bin mapping, volumetric/weight capacity evaluation, velocity-based bin recommendations.	REST / Redis Spatial Cache
FEFO Inventory Ledger	PostgreSQL 16 Stored Triggers / Python	Append-only transaction logging, batch expiry monitoring, FEFO priority queue sorting, automated quarantine locks.	ACID PostgreSQL Database Engine
POS Concurrency Controller	Python FastAPI / asyncpg	High-speed checkout deduction, row-level locking (SELECT ... FOR UPDATE), idempotency token validation.	Direct PostgreSQL connection pool
Replenishment DSS Engine	Python / NumPy / SciPy	Continuous calculation of EOQ, Greasley's Statistical Safety Stock, dynamic ROP alerting, automated draft PO creation.	Background Celery Worker / Cron
Outbound Wave Dispatch	Python FastAPI	Branch requisition consolidation, wave picking batching, shortest-path aisle routing (TSP heuristic), dispatch chalans.	REST / PostgreSQL Transactions
Audit & Anomaly Service	Python / Scikit-learn	Immutable audit logging, cycle count blind entry reconciliation, Isolation Forest machine learning for shrinkage detection.	Asynchronous Celery Tasks

4. End-to-End Sequence Diagrams

4.1 Sequence 1: Inbound Receiving & Digital GRN Generation

```
sequenceDiagram
    autonumber
    actor Clerk as Receiving Clerk
    participant Scanner as Mobile PWA Scanner
    participant Gateway as API Gateway (Nginx)
    participant InboundSvc as Inbound & GRN Service
    participant DB as PostgreSQL Core DB

    Clerk->>Scanner: Selects PO #PO-2026-0891
    Scanner->>Gateway: GET /api/v1/inbound/po/PO-2026-0891
    Gateway->>InboundSvc: Query PO Line Items
    InboundSvc->>DB: SELECT * FROM purchase_orders WHERE po_number = ...
    DB-->>InboundSvc: Returns Expected Items (e.g. 100 Cartons Milk)
    InboundSvc-->>Scanner: Renders PO Checklist UI

    loop For Each Delivered Pallet/Carton
        Clerk->>Scanner: Scans Barcode (EAN-13) + Inputs Expiry Date
        Scanner->>InboundSvc: POST /api/v1/inbound/verify-scan {sku, batch, expiry, qty}
        InboundSvc->>InboundSvc: Validate Minimum Remaining Shelf-Life (>= 75%)
        InboundSvc-->>Scanner: Audio Beep (Green) + Accepted Count Incremented
    end

    opt Damage Detected
        Clerk->>Scanner: Flags 4 cartons as "Damaged" with photo
        Scanner->>InboundSvc: POST /api/v1/inbound/flag-damage {qty: 4, reason: "LEAKING"}
    end

    Clerk->>Scanner: Clicks "Complete Goods Receipt"
    Scanner->>InboundSvc: POST /api/v1/inbound/grn/complete
    InboundSvc->>DB: BEGIN TRANSACTION
    InboundSvc->>DB: INSERT INTO goods_receipt_notes (accepted=96, rejected=4)
    InboundSvc->>DB: INSERT INTO product_batches (status='AVAILABLE', qty=96)
    InboundSvc->>DB: INSERT INTO product_batches (status='QUARANTINED', qty=4)
    InboundSvc->>DB: INSERT INTO inventory_transactions (type='INBOUND_GRN', ...)
    InboundSvc->>DB: UPDATE purchase_orders SET status='COMPLETED'
    InboundSvc->>DB: COMMIT
    InboundSvc-->>Scanner: Returns GRN #GRN-2026-0412 + Auto Credit Note
    Scanner-->>Clerk: Displays GRN Summary & Triggers Pallet Label Print
```

4.2 Sequence 2: Sub-2.0s POS Checkout Atomic Stock Deduction

```
sequenceDiagram
    autonumber
    actor Cashier as Retail Store Cashier
    participant POS as Store POS Terminal
    participant Gateway as API Gateway
    participant POSController as POS Concurrency Controller
```

participant DB as PostgreSQL Core DB

Cashier->>POS: Scans Item Barcode (e.g., 8941100234123)

POS->>Gateway: POST /api/v1/pos/sync
Headers: X-Idempotency-Key: uuid-98234
Payload: {register_code: "REG-01", barcode: "8941100234123", qty: 2}

Gateway->>POSController: Process Atomic Deduction

POSController->>DB: BEGIN TRANSACTION (Isolation: READ COMMITTED)

critical Row-Level Locking on Earliest FEFO Batch

POSController->>DB: SELECT batch_id, current_quantity, expiry_date
FROM product_batches
WHERE product_id = :p_id AND quarantine_status = 'AVAILABLE' AND current_quantity >= :qty
ORDER BY expiry_date ASC LIMIT 1
FOR UPDATE;

DB->>POSController: Returns Batch #B-4091 (Current Qty: 45, Expiry: 2026-10-04)

POSController->>DB: UPDATE product_batches
SET current_quantity = current_quantity - 2
WHERE batch_id = 'B-4091';

POSController->>DB: INSERT INTO inventory_transactions
(product_id, batch_id, type='POS_SALE_FEFO', qty=-2, register_id=...);
end

POSController->>DB: COMMIT;

DB->>POSController: Transaction Committed in 18ms

POSController->>POS: HTTP 200 OK {status: "CONFIRMED", remaining_stock: 43} [Elapsed: 140ms]

POS->>Cashier: Item Added to Customer Bill Instantly

5. Concurrency Control & Transaction Isolation Architecture

5.1 The Concurrency Challenge in Super Shop Retail

During festival rush periods (such as Ramadan afternoon grocery shopping), multiple cash registers across branches and warehouse pickers simultaneously query and decrement stock balances. Naive optimistic locking (`UPDATE ... WHERE version = x`) results in widespread transaction rollback errors on high-velocity items, frustrating cashiers and stalling checkout queues.

5.2 Pessimistic Row-Level Locking Architecture (`SELECT ... FOR UPDATE`)

RetailSync enforces **Pessimistic Row-Level Locking** specifically at the batch row level inside an atomic transaction:

```
-- RetailSync Atomic Deduction Stored Procedure / Query Pattern
BEGIN;
```

```
-- 1. Identify and lock exclusively the earliest active FEFO batch with adequate stock
```

```
SELECT batch_id, current_quantity
FROM product_batches
WHERE product_id = $1
  AND quarantine_status = 'AVAILABLE'
  AND current_quantity >= $2
ORDER BY expiry_date ASC
LIMIT 1
FOR UPDATE;
```

```

-- 2. Decrement physical stock
UPDATE product_batches
SET current_quantity = current_quantity - $2
WHERE batch_id = $selected_batch_id;

-- 3. Append immutable transaction audit record
INSERT INTO inventory_transactions (
  product_id, batch_id, location_id, transaction_type, quantity, reference_document_no, user_id
) VALUES (
  $1, $selected_batch_id, $location_id, 'POS_SALE_FEOF', -$2, $receipt_no, $user_id
);

COMMIT;

```

5.3 Deadlock Elimination Strategy

1. **Deterministic Lock Ordering:** If a composite transaction deducts multiple SKUs (e.g., shopping cart checkout), the application layer strictly sorts all requested SKU IDs in ascending numerical order before acquiring database locks ([ORDER BY product_id ASC](#)).
2. **Short Transaction Life:** Business logic validation (price calculation, customer loyalty) executes *prior* to opening the database transaction. Database transactions remain open for less than **25 milliseconds**.
3. **Connection Pooling Tuning:** PgBouncer maintains dedicated transaction-mode pools ensuring high throughput without exhausting database worker threads.

6. Offline-First Resilience & Network Outage Architecture

```

graph TD
  subgraph ClientEdge["Retail Branch POS / Mobile Scanner Edge"]
    UI["React / PWA UI Layer"]
    SW["Service Worker (Cache Storage)"]
    IDB["Encrypted IndexedDB<br/>Offline Transactions Queue"]
    SyncManager["Background Sync Manager"]
  end

  subgraph Network["Connectivity Layer"]
    ISP["Internet Link (Fiber / 4G)"]
  end

  subgraph CentralServer["RetailSync Central Cloud Server"]
    API["Idempotent POS Ingestion API"]
    IdemCache["Redis Idempotency Cache<br/>(Key: X-Idempotency-Key)"]
    CoreDB["PostgreSQL ACID Core"]
  end

  UI --> |1. Scan Item| SW
  SW --> |2. Check Connectivity| ISP
  ISP -. -> |3a. Connection Failed / Drop| IDB

```

SW --> | 3b. Buffer Transaction | IDB
 UI --> | Visual Amber Badge | UI

SyncManager --> | 4. Detects Link Recovery | IDB
 SyncManager --> | 5. Bulk Replay with Idempotency Tokens | ISP
 ISP --> | POST /api/v1/pos/bulk-sync | API
 API --> | 6. Check Key Exists? | IdemCache
 IdemCache --> | New Key | CoreDB
 CoreDB --> | 7. Commit Batch | API
 API --> | HTTP 200 Success | SyncManager
 SyncManager --> | 8. Clear Buffered Queue | IDB

6.1 Idempotency Guarantee

Every offline transaction generates a unique UUIDv4 token passed via the HTTP header:

X-Idempotency-Key: e4b2d184-7832-4e92-9844-329810a911ab

When bulk-sync replays, Redis verifies whether the idempotency key was previously processed within a 48-hour TTL, completely eliminating duplicate stock deductions even if network retries occur multiple times.

7. Security, RBAC & Audit Architecture

```
graph LR
  subgraph Client["Client Tier"]
    ClientApp["Browser / PWA / Scanner"]
  end

  subgraph SecurityGate["Security & Auth Middleware"]
    TLS["TLS 1.3 Encryption<br/>(Port 443)"]
    JWTVerify["JWT Signature Verification<br/>(HMAC-SHA256 / Ed25519)"]
    RBAC["Role-Based Access Guard<br/>(Roles: Cashier, Clerk, Operator, Sup, Proc, Admin)"]
  end

  subgraph Resources["Protected Domain Endpoints"]
    PublicEnd["Public: /api/v1/auth/login"]
    OperatorEnd["Floor: /api/v1/putaway, /picking"]
    ManagerEnd["Management: /api/v1/po, /adjustments"]
    AdminEnd["Admin: /api/v1/admin, /system"]
  end

  subgraph Audit["Audit Layer"]
    AuditLogger["Append-Only Security & Audit Logger"]
    DBAudit["PostgreSQL<br/>inventory_transactions & security_logs"]
  end

  ClientApp --> TLS
  TLS --> JWTVerify
  JWTVerify --> RBAC
```

```

RBAC --> OperatorEnd
RBAC --> ManagerEnd
RBAC --> AdminEnd
JWTVerify --> PublicEnd

OperatorEnd & ManagerEnd & AdminEnd --> AuditLogger
AuditLogger --> DBAudit

```

7.1 Database Permission Hardening

At the database engine level, the application database user role (`retailsync_app`) is granted `INSERT` and `SELECT` privileges on `inventory_transactions`, but is strictly revoked of `UPDATE` and `DELETE` permissions:

```

REVOKE UPDATE, DELETE ON inventory_transactions FROM retailsync_app;
-- Guaranteed immutable audit ledger by engine constraints

```

8. Hybrid Enterprise Edge Gateway (WarePulse IoT Integration)

For high-throughput central distribution warehouses receiving 50+ pallet deliveries daily, RetailSync integrates the **WarePulse IoT Edge Gateway** as an optional high-speed automated dock intake portal:

```

graph TD
    subgraph DockPortal["Central Distribution Dock Gate Array"]
        Gate["Overhead Fixed Dock Portal"]
        MCU["ESP32 Dual-Core 240MHz Microcontroller"]
        RFID["RC522 High-Frequency RFID Reader / Fixed GS1 Barcode Gun"]
        Flash["LittleFS Non-Volatile Flash Cache (Up to 10,000 Scans)"]
        Status["Audio-Visual Status Beeper / LED"]
    end

    subgraph Telemetry["Encrypted Telemetry Pipeline"]
        MQTT["MQTT Broker (Mosquitto/EMQX)<br/>TLS 1.3, Port 8883, X.509 Client Certs"]
        Topic["Topic: warehouse/dock-01/inbound (QoS 1)"]
    end

    subgraph RetailSyncCore["RetailSync Core Platform"]
        IngestionWorker["Dock Ingestion Worker Service"]
        POEngine["Purchase Order Auto-Reconciler"]
        CoreDB["PostgreSQL 16 DB"]
    end

    Gate --> |Pallet Passes Under Dock| RFID
    RFID --> |SPI Bus| MCU
    MCU --> Status
    MCU --> |Network Brownout| Flash
    Flash --> |On Link Reconnect| MCU
    MCU --> |JSON Payload over MQTTS| MQTT

```

MQTT --> Topic
Topic --> IngestionWorker
IngestionWorker --> POEngine
POEngine --> CoreDB

- **Zero-Touch Inbound Registration:** Pallets pass through loading dock portals; tags are read hands-free within 150 ms; draft GRNs are created automatically in RetailSync with zero manual data entry.